

Week1: Introduction, Secure coding principles and Threat Modelling,

1 About the unit

This unit is for students who already know the basics of coding, and would like to know how to produce code without vulnerabilities. You do not need to be a software developer of many years, or require a significant amount of 'hacking' or information security experience.

The unit will provide VM, Vulnerable web applications and prevention mechanisms (separate web apps). You will use those applications to complete your easement tasks. However, you are recommended to put efforts into learning how to code to take full advantage of the unit material.

The learning outcomes for this unit are as follows:

Table 1: Learning outcomes

ULO1:	Explain secure coding principles to a range of audiences, and apply them to software design and development activities.
ULO2:	Analyse program code and design documentation to identify common programming problems and recommend strategies to mitigate and report issues.
ULO3:	Perform static and dynamic analysis tools and techniques to find vulnerabilities within software.
ULO4:	Use proof techniques and automated theorem provers to formally verify software correctness.

2 Structure and learning outcomes of the course

Goal of the unit

The main goal of this unit is to allow students to learn the cyber security threats that web applications have to deal with. But most cyber threats emerge due to vulnerabilities in the code, hence we will in this unit analyse web application code to understand what coding practices lead to vulnerabilities in them. Furthermore, we will learn how these vulnerabilities can be mitigated to secure the web application. We will be using Java as the primary programming language to discuss these vulnerabilities, however, these vulnerabilities are universal to most programming languages.

- Reference for this note is as below:
- **Threat modelling and secure design:** mostly based on the book *Threat Modeling: Designing for Security*, John Wiley Sons, Incorporated, 2014, by Adam Shostack.
- **Secure coding principles:** based on the OWASP Secure Coding Practices <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>

3 About Software Security

Software security is a sub-discipline of Computer Science. It studies how software can be misused and what is the impact of that unintended behaviour. It also concerns the development of secure software, that is, software that cannot be misused or that doesn't contain an exploitable vulnerability.

Programming errors can occur either while designing or implementing the application. Errors at the implementation level are bugs that can result in unexpected behaviour of the application. But bugs can be detected and removed. Flaws in applications can lead to deeper problems and harder to detect at design stage. We refer to a *vulnerability* to a feature of a software or application that can be exploited by a malicious actor or hacker. The aim of the hacker is to target the confidentiality, integrity or availability of the applications. Most successful hacks rely on a software vulnerability, hence the raising interest on writing secure code and catching software vulnerabilities at the earliest stages of development.

Now, it is worth remarking that application security is not only about secure coding. Coding is just a part of the software life cycle ¹, a very important one, though. Other parts include installation, deployment, maintenance, etc. Those

¹<https://snyk.io/learn/sdlc-software-development-life-cycle/>

fall out of the scope of this course, rather we focus is on secure coding and threat modelling.

4 Why is software vulnerability such a big deal?

Technology is everywhere in every facet of life and software is an important component of it. Many critical systems have their components connected to the Internet and perform critical functions that have direct impact on the economy, livelihoods and health. Vulnerable and insecure technology that support such critical functions can invite cyber intrusions that can have severe consequences.

There is an ongoing explosion of devices needing software, this is especially due to the Internet of Things (IoT) where dedicated devices (instead of multi-purpose computers) are connected directly to the Internet. This poses a growing need for code and software developers, who may face unrealistic time pressure to rush code to production.

Combining the connectivity, complexity, and extensibility of software, we get a comfortably-sized attack surface for the attacker. It is perhaps not surprising, therefore, that the number of vulnerabilities in software has steadily risen over the years. Vulnerabilities and their statistics can be queried from the National Vulnerability Database and from the Common Vulnerability², Exposure database³ and Common Weakness Enumeration (CWE)⁴.

Use the above mentioned CVE database to search and read about the recently discovered CVE-2022-22965 vulnerability.

Many argue that integrating security throughout the entire software development life cycle is not only a necessity, but an opportunity to:

- Comply with regulations, standards or requirements
- Reduce cost by detecting and eliminating flaws at the earliest phases of development
- Improve developer confidence
- Improve customer satisfaction
- Embed a culture of quality and reliability
- Reuse of trusted software in future development
- Reduce maintenance cost

So, if vulnerabilities don't scare you, there is still a business argument in favour of writing secure code.

²<https://nvd.nist.gov/vuln/search>

³https://cve.mitre.org/cve/search_cve_list.html

⁴<https://cwe.mitre.org/>

5 Secure Design Principles: a warm up

Before describing the structure and specific objectives of this course, let's recap principles that are applicable to securing systems in general.

Research activity

Your task is to research about the following security principles that are applied when building secure code.

1. Principle of least privilege
2. Principle of separation of duties
3. Principle of defence in depth
4. Principle of failing secure
5. Principle of open design and avoiding security by obscurity
6. Principle of secure by default / secure by design

References: https://owasp.org/www-project-developer-guide/draft/foundations/security_principles/

By no means the list above is exhaustive. Other principles will be covered in more detail as this course progresses.

6 OWSAP top ten

<https://owasp.org/www-project-top-ten/> The OWASP Top 10 is a standard awareness document for developers and web application security. It represents a broad consensus about the most critical security risks to web applications. (source: <https://owasp.org/www-project-top-ten/>)

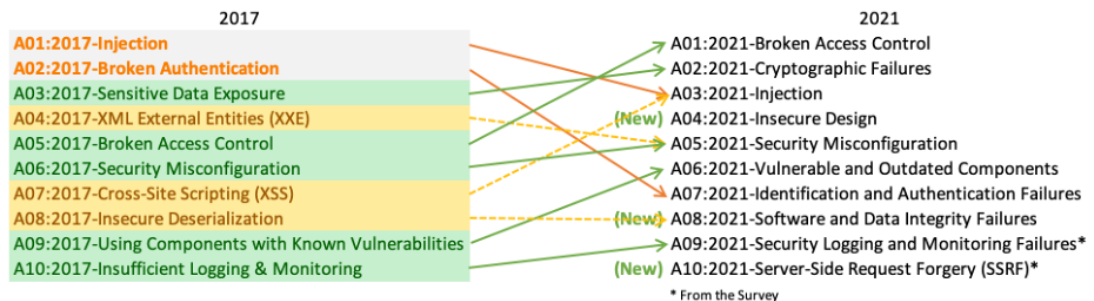


Figure 1: Top 10 Web Application Security Risks. (source: <https://owasp.org/www-project-top-ten/>)